E-Commerce Database Management System

Problem Statement

Develop a database for an E-Commerce platform that manages customers, product categories, products, customer orders, and payment transactions. The system should enable efficient product management, order processing, and payment tracking while generating useful business reports.

Expected Outcome

The system should be able to:

- Manage customer information
- Store product details under different categories
- Record customer orders
- Track payment status
- Generate sales reports
- Identify best-selling products.
- View customer purchase history.

Database Design

- Customers (customer_id PK, customer_name, email, phone, city)
- Categories (category_id PK, category_name)
- Products (product_id PK, category_id FK, product_name, price, stock_quantity)
- Orders (order_id PK, customer_id FK, product_id FK, quantity, order_date, order_status)
- Payments (payment_id PK, order_id FK, payment_method, payment_amount, payment_status, payment_date)

Sample Data

Customers (10 Records)

```
INSERT INTO Customers VALUES

(101,'Aarav Sharma','aarav@gmail.com','9876500001','Hyderabad'),
(102,'Bhavya Reddy','bhavya@gmail.com','9876500002','Vijayawada'),
(103,'Charan Kumar','charan@gmail.com','9876500003','Visakhapatnam'),
(104,'Divya  Patel','divya@gmail.com','9876500004','Bengaluru'),
(105,'Esha  Gupta','esha@gmail.com','9876500005','Chennai'),
(106,'Farhan Ali','farhan@gmail.com','9876500006','Mumbai'),
(107,'Gopi Krishna','gopi@gmail.com','9876500007','Warangal'),
(108,'Harini Rao','harini@gmail.com','9876500008','Guntur'),
(109,'Ishaan Verma','ishaan@gmail.com','9876500009','Pune'),
(110,'John Mathew','john@gmail.com','9876500010','Kochi');
```

Categories (5 Records)

```
INSERT INTO Categories VALUES
(201,'Electronics'), (202,'Fashion'),
(203,'Books'), (204,'Home Appliances'), (205,'Sports');
```

Products (10 Records)

```
INSERT INTO Products VALUES
(301,201,'Laptop',65000,20),
(302,201,'Smartphone',25000,40),
(303,201,'Bluetooth Speaker',3000,50),
(304,202,'Running  Shoes',4500,35),
(305,202,'Backpack',1800,60),
(306,203,'Python Programming',700,100),
(307,203,'SQL Complete Guide',650,80),
(308,204,'Mixer Grinder',4200,25),
(309,204,'Air Fryer',5500,18),
(310,205,'Cricket  Bat',2200,30);
```

Orders (12 Records)

```
INSERT INTO Orders VALUES
(401,101,301,1,'2026-07-01','Delivered'),
(402,102,302,1,'2026-07-02','Delivered'),
(403,103,304,2,'2026-07-03','Shipped'),
(404,104,306,1,'2026-07-04','Delivered'),
(405,105,309,1,'2026-07-05','Pending'),
(406,106,303,3,'2026-07-06','Delivered'),
(407,107,307,2,'2026-07-07','Delivered'),
(408,108,305,1,'2026-07-08','Cancelled'),
(409,109,308,1,'2026-07-09','Delivered'),
(410,110,310,2,'2026-07-10','Shipped'),
(411,101,302,1,'2026-07-11','Delivered'),
(412,103,301,1,'2026-07-12','Pending');
```

Payments (12 Records)

```
INSERT INTO Payments VALUES
(501,401,'UPI',65000,'Paid','2026-07-01'),
(502,402,'Credit    Card',25000,'Paid','2026-07-02'),
(503,403,'Debit    Card',9000,'Paid','2026-07-03'),
(504,404,'UPI',700,'Paid','2026-07-04'),
(505,405,'Net    Banking',5500,'Pending','2026-07-05'),
(506,406,'UPI',9000,'Paid','2026-07-06'),
(507,407,'Credit    Card',1300,'Paid','2026-07-07'),
(508,408,'Cash on Delivery',1800,'Cancelled','2026-07-08'),
(509,409,'UPI',4200,'Paid','2026-07-09'),
(510,410,'Debit    Card',4400,'Paid','2026-07-10'),
(511,411,'UPI',25000,'Paid','2026-07-11'),
(512,412,'Credit    Card',65000,'Pending','2026-07-12');
```

SQL Questions

1. Display all products along with their prices and available stock.
2. Increase the price of all products in a particular category by 5%.
3. Change the payment status of a pending payment to **Paid**.
4. Delete a cancelled order.
5. Display all customer names in uppercase and lowercase.
6. Display the first five characters of each product name.
7. Concatenate the customer name and city.
8. Replace the word **Guide** with **Handbook** in product names.
9. Display last week orders.
10. Find the number of days since each order was placed.
11. Display the payment date in **DD-Mon-YYYY** format.
12. Display the month and year of every order.
13. Count the total number of products in each category.
14. Display the average product price for each category.
15. Find the highest-priced product.
16. Display the total sales amount collected through each payment method.
17. Find the average quantity ordered for each product.
18. Display customer names, product names, and order dates.
19. Display customer names, ordered products, and payment status.
20. Display customer names, product names, quantity ordered, and payment amount.
21. Display all products along with their category names, even if no orders have been placed for them.
22. Display all customers and their order details.
23. Display category names, product names, customer names, and order status in a single result.
24. Find customers whose payment amount is greater than the average payment amount.
25. Display the category that contains the most expensive product.
26. Find customers who purchased the highest-priced product.
27. Display products whose price is greater than the average price of all products.
28. Find customers who have never placed an order using NOT EXISTS.
29. Create a view named Customer_Order_Details displaying customer name, product name, quantity, order date, and payment status.
30. Create another view named Category_Sales_Report displaying category name, total products sold, and total sales amount.
31. Retrieve records from both views.
32. Start a transaction. Update the stock quantity of a product. Create a SAVEPOINT. Then update the payment status of an order. Roll back to the savepoint. Commit the transaction.
33. Create a stored procedure to display products whose stock quantity is below a given value.
    - Input: stock_limit
    - Display product name, price, and stock quantity.
34. Create a stored procedure to find the total number of orders placed by

customers from a particular city.

- Input: city
- Display the number of orders.

35. Create a stored procedure to display the details of a particular order.
    - Input: order_id
    - Display customer name, product name, quantity, order date, order status, and payment status.
36. Create a trigger on the Products table.

    If someone tries to set stock_quantity to a negative value, prevent the update.
37. Create a trigger on the Orders table.

    Whenever a new order is placed, automatically reduce the corresponding product's stock_quantity based on the ordered quantity.